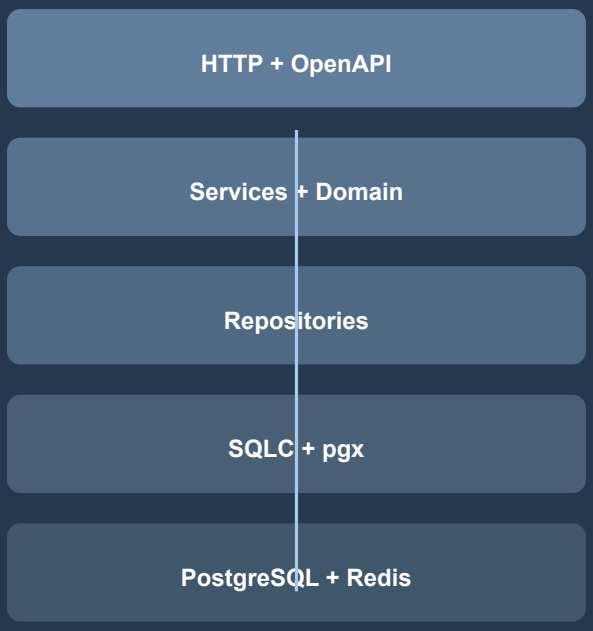
UNDERSTANDING THE

FortyOne API Platform

A visual, example-driven guide from the system mental model down to Go, SQLC, security, integrations, workers, and tests.

Progressive depth: orientation -> request trace -> implementation mechanics



Based on the implemented API modernization snapshot - 28 August 2026

How to use this guide

This book is designed for two kinds of reading. The first pass gives you a reliable mental model without requiring deep Go knowledge. The second pass explains the mechanics closely enough that you can trace, review, and eventually change the platform safely.

Pass	Goal	Read
1 - Orientation	Explain the platform to another person	Parts I and II, diagrams, and the chapter summaries
2 - Engineering	Trace code and understand design decisions	Parts III through VI and the worked story example
3 - Practice	Make a safe, complete change	Part VII, exercises, source map, and command reference

> **The key learning rule**
> Do not memorize every package. Learn the direction of responsibility: transport translates, services decide, repositories persist, adapters integrate, and bootstrap connects everything.

What this guide covers

- The API and worker as two Go processes with shared platform foundations.
- The modular-monolith structure and how to locate any behavior.
- A complete public story-creation request from HTTP to PostgreSQL and back.
- SQLC, pgx, transactions, outboxes, idempotency, and pagination.
- Actors, scopes, roles, tenant boundaries, credentials, OAuth, and webhooks.
- Provider-neutral integrations and the path for GitLab or future adapters.
- Testing layers, quality gates, operations, and how to make a change.

What this guide does not claim

The local code migration and hermetic checks are complete. This guide does not claim production acceptance. Database-backed migration tests, realistic query plans, load tests, live provider exercises, staging drills, and production rollout remain separate acceptance work.

Contents

PART I

The mental model

Start with the platform as a whole: what runs, what each layer owns, and where to look.

1. The platform in one picture

FortyOne is a modular monolith. That means the business capabilities live in one deployable Go application, but they are separated internally as if each were a small product with its own vocabulary, rules, persistence, and transport.

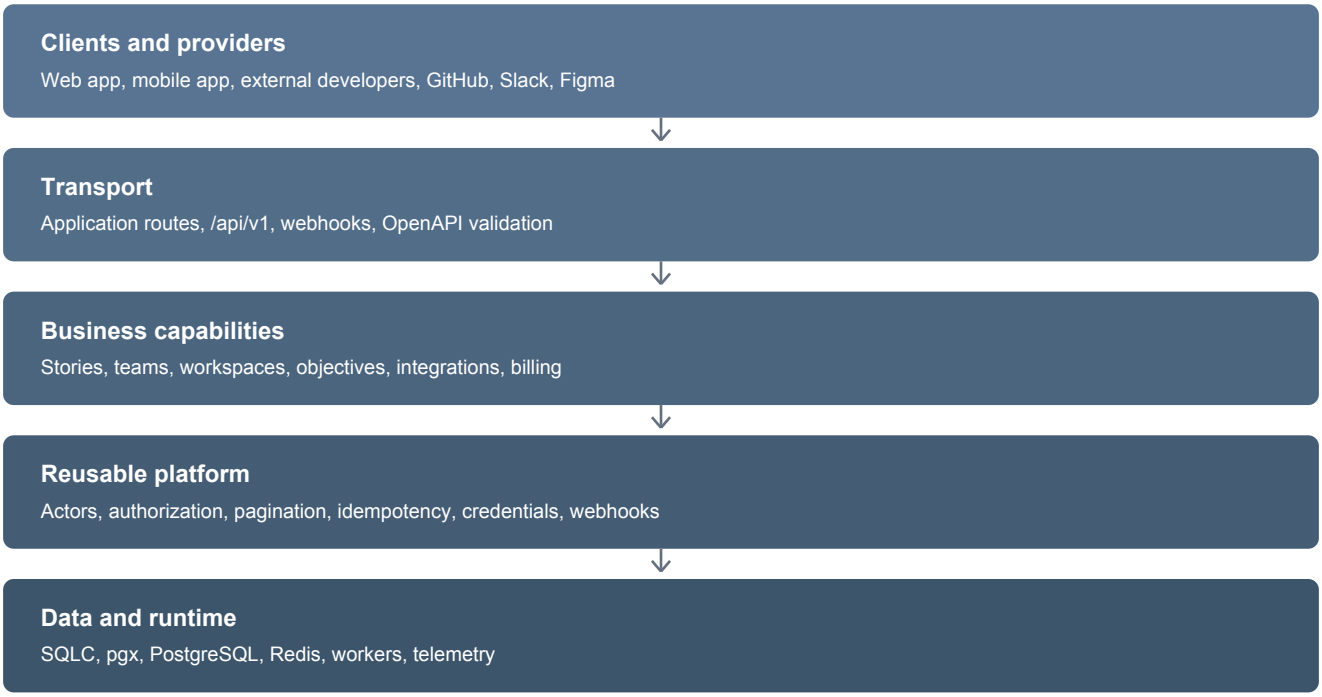


Figure 1. The system is layered, but business behavior is grouped by capability rather than by one giant global controller.

Why a modular monolith?

- One deployment is simpler than coordinating many microservices.
- Module boundaries still create clear ownership and prevent a single ball of code.
- Transactions can safely protect invariants inside one PostgreSQL database.
- A future service extraction is possible only when a real scaling or ownership need appears.

> **Beginner translation**
> A module is a department. HTTP is reception, the service is the decision maker, the repository is records management, and bootstrap is the building manager connecting departments to shared utilities.

The two primary processes

Process	Owns	Typical work
API - cmd/api	HTTP, SSE, public API, inbound requests	Authenticate, authorize, execute a use case, return a response
Worker - cmd/worker	Queues, schedules, retries, provider delivery	Load durable work, lease it, execute safely, record outcome

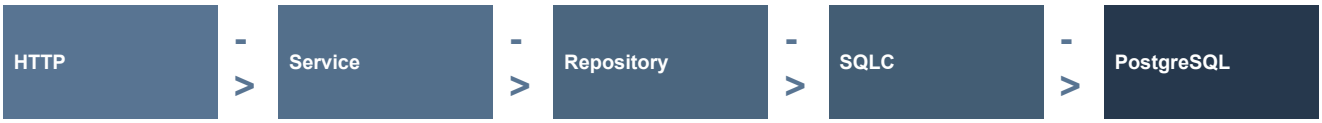
2. Where everything lives

The most important navigation rule is that ownership follows the business capability. If you are changing stories, begin in the stories module. Do not begin by searching for a global SQL folder or a global service file.

```
apps/server/
  cmd/api/                          API composition and entry point
  cmd/worker/                       worker composition and entry point
  internal/bootstrap/              dependency wiring and runtime ownership
  internal/modules/<capability>/
    domain/                         stable business vocabulary
    http/                           transport translation
    service/                        use cases and policy
    repository/                     persistence adapter
      queries/*.sql                 handwritten named SQLC queries
      sqlc/                         generated Go code - do not hand edit
  internal/platform/               cross-cutting reusable primitives
  api/openapi/v1/                   public API source contract
  docs/                             architecture, database, security, onboarding
```

The canonical directory map.

The dependency direction



Calls flow to the right. Domain values and narrow interfaces allow policy to remain independent of generated database types or provider SDKs.

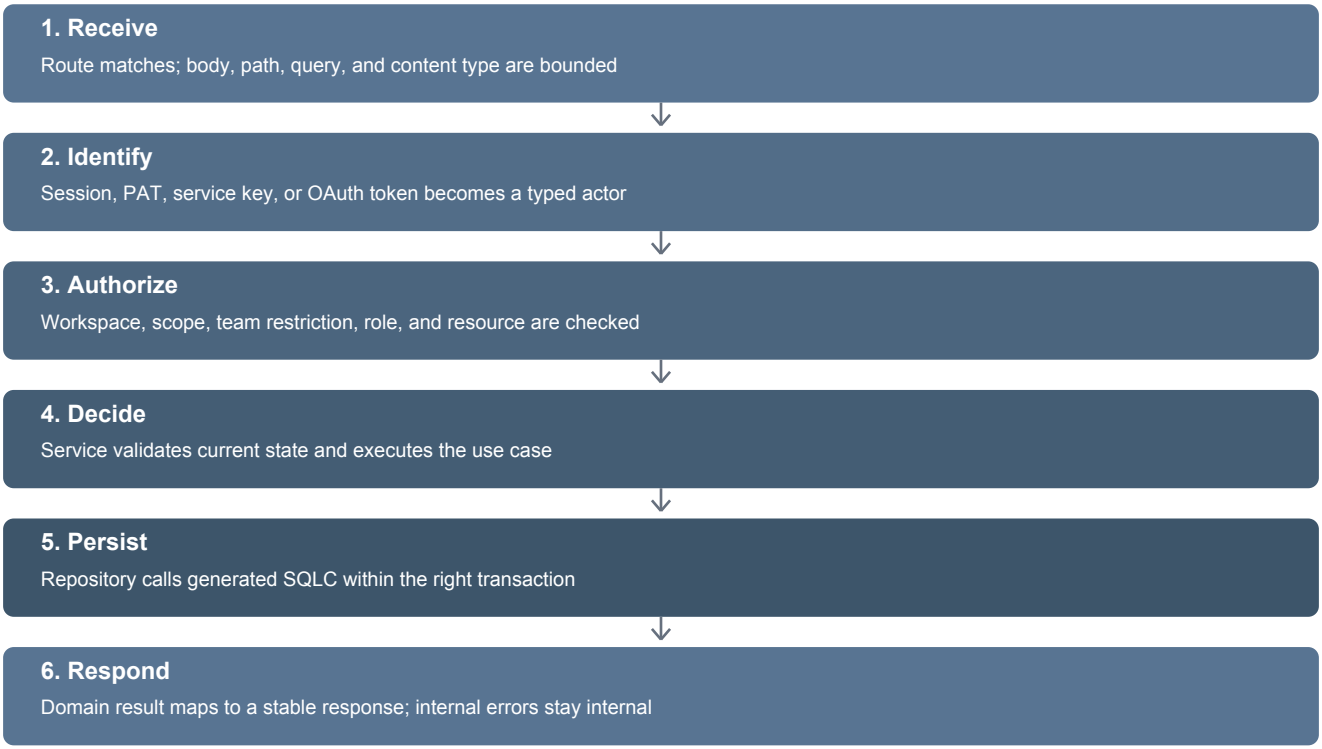
What is deliberately forbidden

Shortcut	Why it is dangerous	Correct home
SQL in a handler	Couples protocol and persistence; authorization is easy to skip	Named SQLC query in the owning repository
Service imports a concrete repository	Makes business logic hard to test and replace	Caller-owned narrow interface
Provider SDK type in domain	Slack or GitHub becomes the business model	Adapter translates to provider-neutral values
Another pagination helper	Creates incompatible list behavior	Shared pagination/cursor package
Generated SQLC model returned by HTTP	Database shape becomes a public contract	Map generated row -> domain -> response DTO

> **Architecture as executable policy**
> The repository includes checks that parse imports and production code. A new dependency leak, direct SQL path, SQLx import, or oversized production file fails the architecture gate.

3. A request from beginning to end

Every request is a sequence of translations and decisions. The same shape applies to the application API and the public API, although the authentication method and response contract differ.



A crucial distinction

Layer	Question it answers
Authentication	Who or what presented this credential?
Authorization	May that actor perform this operation on this resource now?
Validation	Is the request structurally and semantically meaningful?
Business policy	Is this transition allowed from current state?
Persistence	How is the approved state read or written atomically?

Defense in depth

Middleware can reject early, but the service remains authoritative. SQL also includes workspace and resource predicates where the database can enforce the boundary.

PART II

Go foundations used by the platform

Understand the language ideas behind the architecture: interfaces, composition, context, errors, and concurrency.

4. Interfaces and dependency inversion

Go interfaces describe behavior, not inheritance. In this platform, the consumer owns the smallest interface it needs. That keeps a service focused and prevents it from depending on an entire concrete implementation.

```
// Owned by the service that needs story persistence.
type StoryStore interface {
    Get(ctx context.Context, workspaceID, storyID uuid.UUID) (Story, error)
    Create(ctx context.Context, command CreateStoryCommand) (Story, error)
}

type Service struct {
    stories StoryStore
    clock   Clock
}

func New(stories StoryStore, clock Clock) *Service {
    return &Service{stories: stories, clock: clock}
}
```

Simplified example: the service depends on capabilities, not a database implementation.

Why the interface is small

- Tests can supply a tiny fake without setting up PostgreSQL.
- The service cannot accidentally use unrelated repository methods.
- A new adapter can satisfy the same contract without changing the use case.
- Compile-time assertions prove important adapters still satisfy their contracts.

```
// Fails at compile time if *stories.Service loses or changes a required method.
var _ messaging.StoryMutationService = (*stories.Service)(nil)
```

A real pattern added after startup exposed an interface mismatch.

> **What happened during startup**
> Messaging expected a comment type owned by the comments module, while the stories service returned its own caller-facing comment type. A runtime assertion failed. The contract now uses the correct stories-domain type and a compile-time assertion prevents recurrence.

Composition at bootstrap

Constructors do not find global dependencies. Bootstrap creates the PostgreSQL pool, repositories, services, and handlers explicitly. This is dependency injection without a magic container.

5. Context, cancellation, and graceful shutdown

A `context.Context` carries cancellation and deadlines across a request or background operation. It is the answer to: 'should this work still continue?'

```
func (s *Service) Get(ctx context.Context, id uuid.UUID) (Story, error) {
    row, err := s.queries.GetStory(ctx, id)
    if err != nil {
        return Story{}, fmt.Errorf("get story %s: %w", id, err)
    }
    return mapStory(row), nil
}
```

The same context reaches PostgreSQL, so cancellation can stop blocked I/O.

API lifecycle

1. Configuration and security keys are parsed and validated.
2. PostgreSQL and Redis are connected and pinged.
3. HTTP, SSE, and the Redis stream consumer initialize before readiness.
4. A root context supervises long-lived components.
5. Shutdown marks readiness draining, cancels work, waits, flushes telemetry, then closes dependencies once.

Readiness versus liveness

Probe	Meaning	Example failure
Liveness	The process is alive	Deadlock or crashed process
Readiness	The process is safe to receive traffic	Starting, draining, PostgreSQL down, Redis down

> **Current local evidence**
> The API now builds, validates configuration, connects to PostgreSQL and Redis, initializes dependencies, and reaches 'API is ready' on port 8000.

6. Errors: useful internally, safe externally

Go makes errors explicit. The platform wraps errors with operational context while keeping public responses stable and free of secrets or internal database details.

```
row, err := queries.GetStory(ctx, params)
switch {
case errors.Is(err, pgx.ErrNoRows):
    return Story{}, ErrNotFound
case err != nil:
    return Story{}, fmt.Errorf("get story: %w", err)
default:
    return mapStory(row), nil
}
```

Repository errors are classified before leaving the persistence boundary.

Two views of one failure

Audience	Receives
Internal log	Operation, request ID, safe structured fields, wrapped cause
API consumer	Stable code, safe message, request ID, optional field violations

```
{
  "error": {
    "code": "invalid_reference",
    "message": "A referenced resource is unavailable in this workspace.",
    "requestId": "req_..."
  }
}
```

The consumer can handle the code; the message reveals no cross-tenant detail.

Panics are not ordinary control flow

Expected failures return errors. Panics indicate broken program invariants. The startup mismatch fixed in this session showed why bootstrap construction should return an error or be guarded by compile-time contracts rather than discovering compatibility after the process starts.

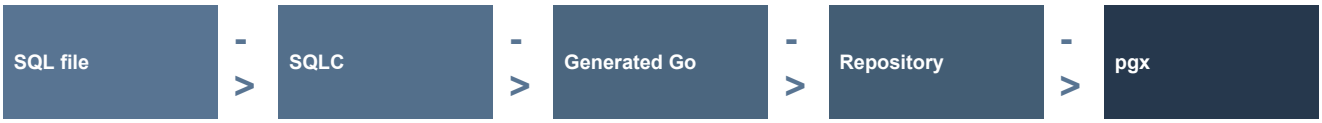
PART III

Typed data and SQLC

Move from SQL text to generated Go types, then understand mapping, transactions, migrations, and pagination.

7. SQLC and pgx: the persistence pipeline

SQLC does not replace SQL. Engineers still write SQL, but SQLC reads it together with the schema and generates Go methods, parameter structs, and result structs. pgx is the runtime PostgreSQL driver and connection pool.



Tool	Responsibility	Not responsible for
PostgreSQL	Constraints, indexes, transactions, query execution	Go API design
SQLC	Compile-time query shapes and generated typed methods	Business authorization or query performance
pgx	Connections, pooling, transactions, PostgreSQL protocol	Mapping database rows to public responses
Repository	Call SQLC, own transactions, map errors and rows	HTTP parsing

A small query example

```
-- name: GetStory :one
SELECT id, title, workspace_id, team_id, created_at
FROM public.stories
WHERE id = sqlc.arg(story_id)
  AND workspace_id = sqlc.arg(workspace_id)
  AND deleted_at IS NULL;
```

Handwritten SQL: named, explicit columns, and tenant-scoped.

```
row, err := queries.GetStory(ctx, storysql.GetStoryParams{
    StoryID:     storyID,
    WorkspaceID: workspaceID,
})
```

Generated call: incorrect field names or types fail compilation.

What type safety buys

Renaming a parameter, changing a result column, or passing a string where PostgreSQL expects a UUID becomes a generation or compile failure instead of a hidden production row-scan error.

What type safety does not buy

SQLC cannot prove that a query is fast, that every business authorization rule is correct, or that production data has the expected distribution. Those require tests, EXPLAIN plans, constraints, and review.

8. The repository boundary

Generated types are database-facing implementation details. A repository converts them into stable domain values and translates PostgreSQL errors into errors the service understands.

```
func (r *repo) PrepareStoryMutation(
    ctx context.Context,
    scope domain.MutationScope,
    teamID uuid.UUID,
) (domain.MutationPreconditions, error) {
    if !scope.Actor.TeamAccess.Allows(teamID) {
        return domain.MutationPreconditions{}, domain.ErrMutationForbidden
    }

    row, err := r.queries.AuthorizeStoryCreate(ctx, sqlc.AuthorizeStoryCreateParams{
        TeamID: teamID, WorkspaceID: scope.WorkspaceID,
        ActorKind: string(scope.Actor.Kind), ActorID: scope.Actor.PrincipalID,
    })
    if errors.Is(err, pgx.ErrNoRows) {
        return domain.MutationPreconditions{}, domain.ErrMutationForbidden
    }
    if err != nil {
        return domain.MutationPreconditions{}, fmt.Errorf("authorize story creation: %w", err)
    }
    return mapPreconditions(row), nil
}
```

Adapted from the real stories repository.

Why mapping matters

- The database schema can evolve without becoming the public API.
- Nullable PostgreSQL values become intentional domain options.
- Generated code can be regenerated freely because handwritten policy is elsewhere.
- Services and tests use business vocabulary rather than SQLC implementation names.

The zero-SQLx result

The migration produced 41 SQLC generation units and 1,218 named SQLC operations, with zero production SQLx imports and no SQLx module dependency. Static application SQL now belongs to a module or platform persistence package.

8A. Advanced SQL patterns used by the platform

The difficult queries are not difficult for decoration. They encode tenant boundaries, concurrency, stable pagination, partial updates, and recovery in the place that can enforce them atomically. This chapter explains the recurring patterns.

Pattern 1 - authorization inside the statement

```
INSERT INTO public.stories (id, workspace_id, team_id, status_id, title)
SELECT
    sqlc.arg(story_id),
    sqlc.arg(workspace_id),
    sqlc.arg(team_id),
    sqlc.narg(status_id),
    sqlc.arg(title)
WHERE EXISTS (
    SELECT 1
    FROM public.team_members tm
    WHERE tm.team_id = sqlc.arg(team_id)
      AND tm.user_id = sqlc.arg(actor_id)
)
AND (
    sqlc.narg(status_id) IS NULL
    OR EXISTS (
        SELECT 1 FROM public.statuses s
        WHERE s.status_id = sqlc.narg(status_id)
          AND s.team_id = sqlc.arg(team_id)
    )
)
RETURNING id, workspace_id, team_id, title;
```

An INSERT ... SELECT can refuse the write when authority or a referenced resource is invalid.

If a protected predicate is false, the statement returns no row. The repository maps that result to a safe forbidden or invalid-reference error. This closes the race between checking a reference and inserting it.

Pattern 2 - CTEs make multi-stage reasoning explicit

```
WITH target AS (
    SELECT s.id, s.workspace_id, s.team_id, s.updated_at
    FROM public.stories s
    WHERE s.id = sqlc.arg(story_id)
      AND s.workspace_id = sqlc.arg(workspace_id)
      AND s.deleted_at IS NULL
), authorized AS (
    SELECT target.*
    FROM target
    WHERE EXISTS (
        SELECT 1 FROM public.team_members tm
        WHERE tm.team_id = target.team_id
          AND tm.user_id = sqlc.arg(actor_id)
    )
)
UPDATE public.stories s
SET title = sqlc.arg(title), updated_at = sqlc.arg(now)
FROM authorized
WHERE s.id = authorized.id
  AND authorized.updated_at = sqlc.arg(expected_updated_at)
RETURNING s.id, s.updated_at;
```

CTEs name the stages: locate tenant-scoped target, authorize it, then compare-and-swap update.

A common table expression is not automatically faster. Its main value here is reviewability: each stage has one grain and one responsibility. Query plans still require measurement.

Pattern 3 - tri-state patch semantics

For nullable fields, an API patch needs three meanings: omitted, explicitly clear, and set to a value. A pointer alone cannot always distinguish all three after transport mapping, so SQL receives both a set flag and a nullable value.

```
UPDATE public.stories
SET
    assignee_id = CASE
        WHEN sqlc.arg(set_assignee_id) THEN sqlc.narg(assignee_id)
        ELSE assignee_id
    END,
    sprint_id = CASE
        WHEN sqlc.arg(set_sprint_id) THEN sqlc.narg(sprint_id)
        ELSE sprint_id
    END,
    updated_at = sqlc.arg(updated_at)
WHERE id = sqlc.arg(story_id)
  AND workspace_id = sqlc.arg(workspace_id)
RETURNING id, assignee_id, sprint_id, updated_at;
```

set_assignee_id=false means unchanged; true + NULL means clear; true + UUID means replace.

Pattern 4 - keyset pagination

```
SELECT id, created_at, title
FROM public.stories
WHERE workspace_id = sqlc.arg(workspace_id)
  AND (created_at, id) < (sqlc.arg(cursor_time), sqlc.arg(cursor_id))
ORDER BY created_at DESC, id DESC
LIMIT sqlc.arg(page_size);
```

The unique tie-breaker ID makes ordering deterministic when timestamps match.

Pattern 5 - claim work without worker collisions

```
WITH candidates AS (
    SELECT id
    FROM public.webhook_deliveries
    WHERE status = 'pending'
      AND next_attempt_at <= sqlc.arg(now)
    ORDER BY next_attempt_at, id
    FOR UPDATE SKIP LOCKED
    LIMIT sqlc.arg(batch_size)
)
UPDATE public.webhook_deliveries d
SET status = 'leased',
    lease_token = sqlc.arg(lease_token),
    lease_expires_at = sqlc.arg(lease_expires_at)
FROM candidates
WHERE d.id = candidates.id
RETURNING d.id, d.endpoint_id, d.attempt_count, d.lease_token;
```

SKIP LOCKED lets replicas claim different rows without waiting on one another.

Pattern 6 - idempotent insert with ON CONFLICT

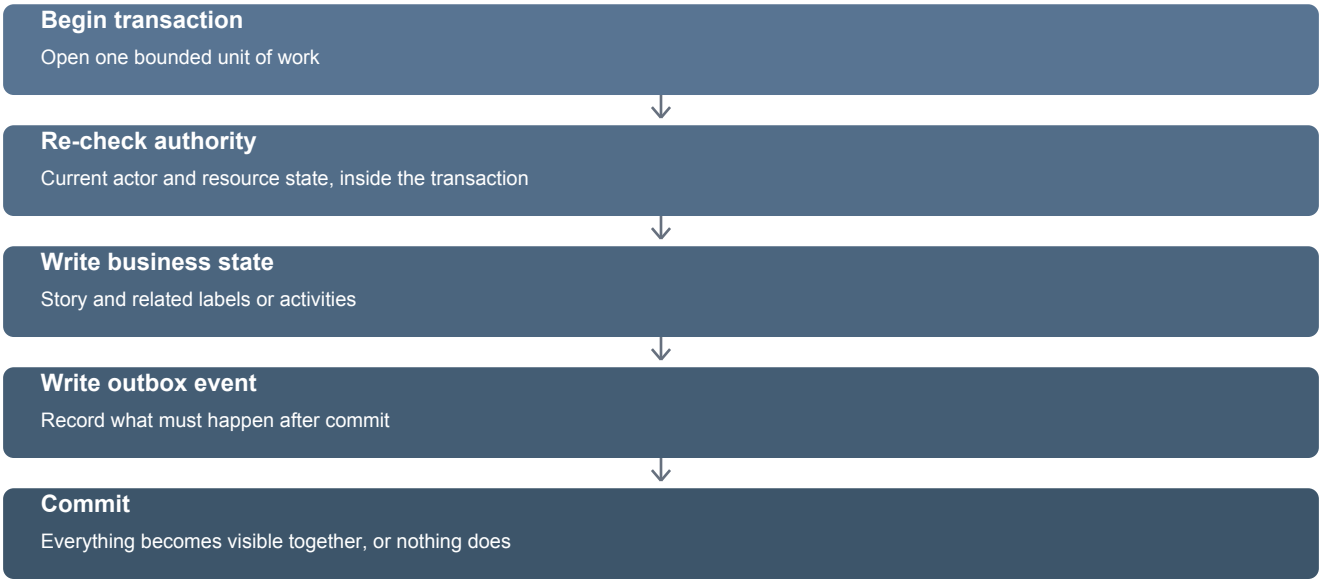
```
INSERT INTO public.webhook_inbox (provider, delivery_id, payload_ciphertext)
VALUES (sqlc.arg(provider), sqlc.arg(delivery_id), sqlc.arg(payload_ciphertext))
ON CONFLICT (provider, delivery_id) DO NOTHING
RETURNING id;
```

A uniqueness constraint is the final concurrency authority; application pre-checks alone are racy.

> **Review checklist for complex SQL**
>
> State the row grain, tenant predicate, authorization predicate, unique order, null meaning, lock behavior, transaction owner, expected index, empty-result meaning, and tests. If these cannot be explained, the query is not ready.

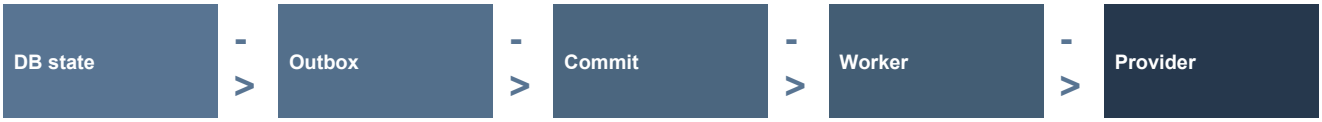
9. Transactions, isolation, and the outbox

A transaction groups database changes into one all-or-nothing unit. The stories mutation path uses serializable transactions for invariants such as sequence allocation, authorized references, activity recording, and event creation.



Why not call GitHub inside the transaction?

A network call may be slow, time out, or succeed while the database later rolls back. Holding database locks during provider I/O also reduces throughput. The platform commits an outbox record with business state, then a worker performs the external delivery.



Retrying serializable transactions

```
for attempt := 1; attempt <= 3; attempt++ {
    err := transactor.WithinTransaction(ctx, serializable, func(tx pgx.Tx) error {
        // authorize, allocate sequence, write story, activity, and event
        return nil
    })
    if err == nil { return result, nil }
    if !database.IsRetryableTransactionError(err) { return result, err }
}
```

Simplified from story creation. Retries are bounded, classified, and observable.

> **Invariant**
>
> State and the event describing that state commit together. Delivery may be later, but the intent is never lost in the gap between commit and network I/O.

10. Migrations and schema evolution

Migrations change the durable shape of production data. They are operational events, not just SQL files. The branch introduces migrations 000152 through 000175 and treats every one already added as immutable.

Rule	Reason
Never edit an applied migration	Environments may already have recorded its checksum and effects
Use a new forward migration	History remains auditable and every environment converges
Document mixed-version behavior	API and worker versions may overlap during rollout
Separate schema presence from backfill	Large data rewrites may need controlled batches
Verify rollback or forward-fix strategy	Not every destructive change can be safely reversed

The safe sequence


```
make migrate-create name=add_capability
make migrate-up
make migrate-version
make sqlc-generate
make migration-check
```

Representative development commands. Production execution requires a separate approved rollout.

> **Current boundary**
> The migration files and contracts are locally verified, but this work does not claim that migrations 000152-000175 were applied to production or exercised on representative production data.

Indexes are hypotheses until measured

The migration adds keyset and automation indexes, but PostgreSQL chooses plans based on statistics and data shape. Production acceptance still requires `EXPLAIN (ANALYZE, BUFFERS)` and realistic load.

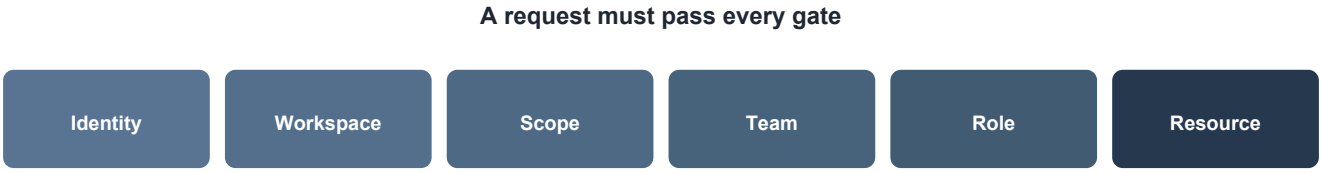
PART IV

Security from identity to storage

See authorization as a chain of explicit gates, then understand credentials, cryptography, OAuth, webhooks, and idempotency.

11. The actor and authorization model

An actor is the authenticated principal plus the restrictions carried by its credential. It is not always a user. The platform supports human sessions, personal access tokens, service accounts, and OAuth applications with different permitted behaviors.



Failure at any gate returns a safe denial; unknown state never becomes permission.

The authorization sequence

1. Validate that the actor is internally coherent and active.
2. Require an explicitly allowed principal kind for the operation.
3. Require the actor and resource to belong to the same workspace.
4. Require every credential scope, such as stories:write.
5. Apply any team restriction carried by the credential.
6. Load current membership and role rather than trusting historical token data.
7. Check resource-specific visibility or ownership.

```
decision := authorization.EvaluateWorkspace(authorization.WorkspacePolicyInput{
    Actor: actor,
    WorkspaceID: story.WorkspaceID,
    WorkspaceRole: currentRole,
    MinimumWorkspaceRole: authorization.RoleMember,
    RequiredScopes: []auth.Scope{auth.ScopeStoriesWrite},
    TeamID: story.TeamID,
    AllowedPrincipalKinds: []auth.PrincipalKind{
        auth.PrincipalUser,
        auth.PrincipalServiceAccount,
        auth.PrincipalOAuthApplication,
    },
})
```

Simplified from the shared authorization policy.

> **Why explicit principal kinds matter**
>
> If a new credential type is added later, it receives no permissions merely because it authenticated successfully. Each operation must deliberately admit it.

12. Hashes, HMACs, and encryption

These mechanisms solve different problems. Treating them as interchangeable creates security defects.

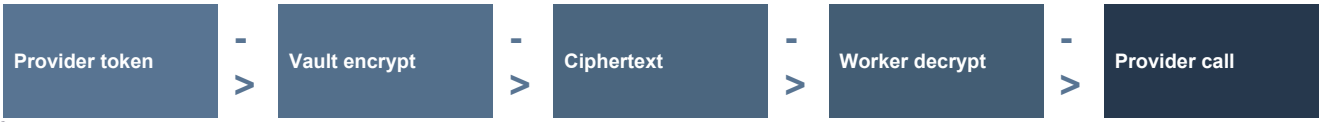
Mechanism	Purpose	FortyOne example
Hash	One-way fingerprint	Request body or idempotency-key identity
HMAC	One-way keyed authenticity check	Stored API-token digest or signed token verification
Encryption	Recoverable confidentiality	Provider access token needed for a future API call
Signature	Authenticity and integrity across systems	Inbound or outbound webhook verification

Why API tokens are not encrypted for lookup

A personal token or service key is shown once. The server stores a keyed digest and compares a digest of the presented token. A database leak does not reveal the original token.

Why provider tokens are encrypted

The server must recover a GitHub, Slack, or Figma token to call that provider later. The credential vault uses versioned keys and context binding. The database stores ciphertext plus metadata, never reusable plaintext.



> **Purpose-specific keys**
> Authentication cookies, verification tokens, invitations, developer credentials, OAuth tokens, messaging confirmations, and webhook payloads use separate keys. Reusing one secret makes one compromise affect unrelated systems.

Key versioning

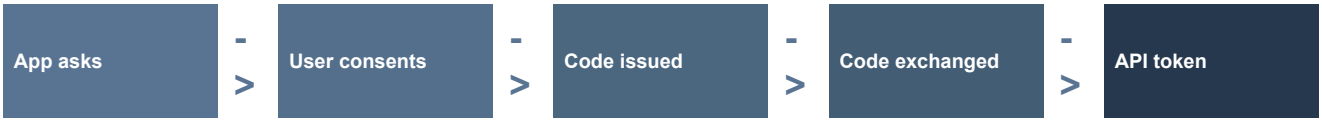
Stored ciphertext records which key version encrypted it. A new active version handles new writes while older versions remain available for reads during controlled rotation. The configuration parser now uses a custom positive key-version type because the previous third-party parser silently ignored unsigned integers.

13. Developer credentials and OAuth

External applications should not borrow browser cookies or pretend to be a normal user. The platform now provides explicit machine and delegated credential forms.

Credential	Represents	Best for
Personal access token	A human user with current membership and scopes	Personal scripts and developer tools
Service-account key	A workspace-owned non-human principal	Backend automation with explicit resource access
OAuth authorization code + PKCE	A user granting an application delegated access	Third-party applications acting for users
OAuth client credentials	An installed confidential application principal	Approved server-to-server story creation

OAuth in plain language



- The authorization code is short-lived and single-use.
- PKCE binds the exchange to the client that initiated it.
- The token audience is exactly the public API resource, not MCP or internal routes.
- Refresh tokens rotate; reuse can revoke the token family.
- Current workspace membership and resource authorization are checked on every use.

Least privilege

A scope such as `stories:write` grants a capability category, not universal access. Workspace binding, team restrictions, principal kind, current role, and resource rules still apply.

```
Authorization: Bearer <token>
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
Content-Type: application/json
```

Representative public API headers. Real credentials must never appear in logs or documentation.

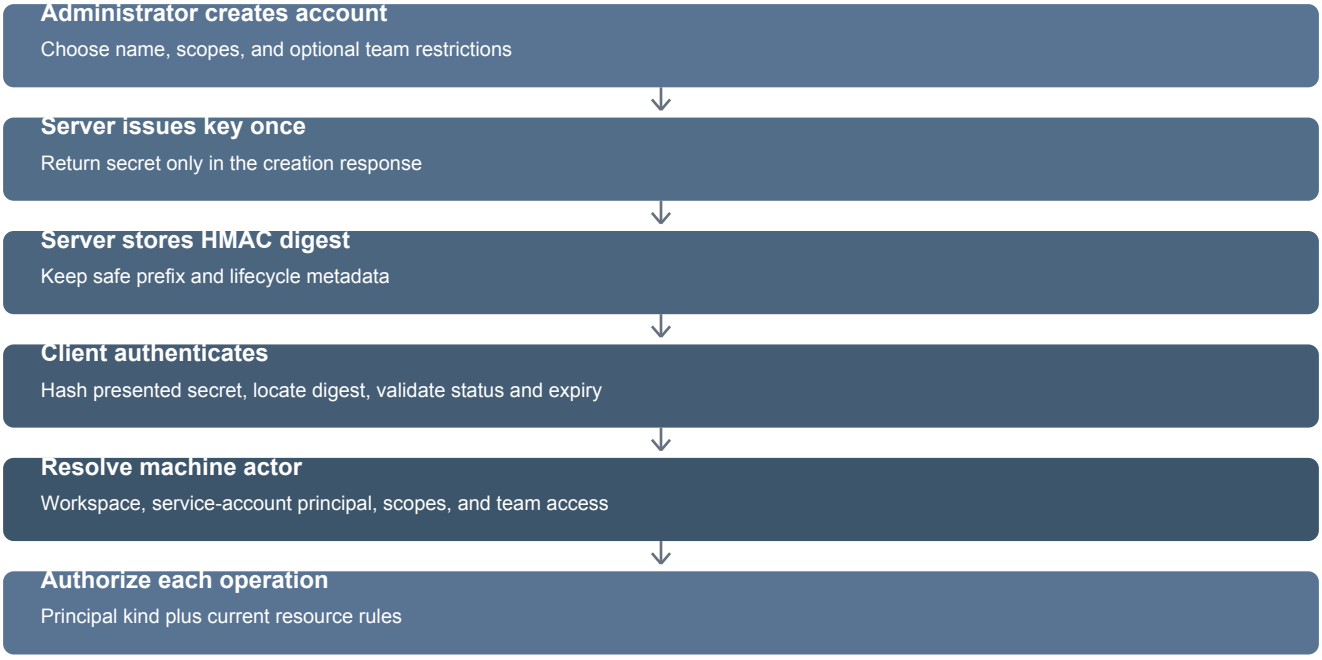
13A. Service accounts in detail

A service account is a workspace-owned machine principal. It is not a user with a hidden email address, and it must never inherit the permissions of the administrator who created it.

The objects involved

Object	Purpose	Security property
Service account	Stable non-human principal in one workspace	Has its own identity and lifecycle
Credential record	Metadata for one issued key	Stores prefix, digest, status, timestamps - not plaintext
Scope grants	Allowed API capabilities	Least privilege and explicit review
Team restrictions	Optional subset of workspace teams	A broad scope cannot escape the team fence
Audit actor	Identity recorded on actions	Never attributes automation to the creator

Creation and use



Conceptual token shape

```
fo_sa_<public-key-id>_<secret-material>
      |                   |
      |                   +-- shown once; never stored in plaintext
      +-- safe lookup prefix; not sufficient to authenticate
```

The exact production format is an implementation contract. This illustrates public lookup plus secret verification.

Rotation without downtime

1. Create a second active credential with the same or narrower policy.
2. Deploy the new key to the external workload through its secret manager.
3. Observe successful use of the new credential identity.
4. Revoke the old credential and verify it fails immediately.
5. Retain only non-sensitive audit metadata.

Never do this

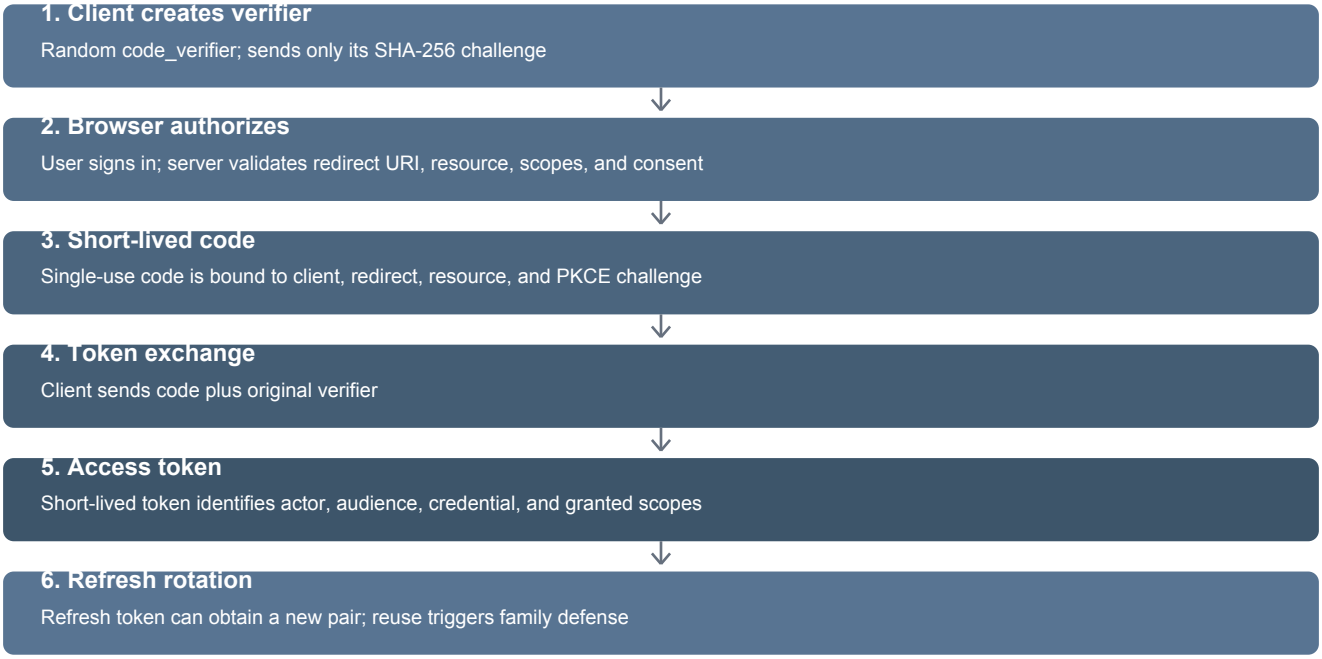
Do not store service-account keys in source control, CI logs, screenshots, browser local storage, tickets, or database plaintext. Do not reuse one key across unrelated workloads; separate identities make revocation and audit meaningful.

13B. OAuth 2.0 in detail

OAuth separates four roles: the resource owner, the client application, the authorization server, and the resource server. In FortyOne, the API is the resource server and accepts tokens only for its exact API audience.

Role	FortyOne example
Resource owner	The user granting access to their FortyOne workspace capabilities
Client	A third-party web, desktop, CLI, or backend application
Authorization server	Issues codes and tokens after authentication and consent
Resource server	https://api.fortyone.app/api/v1

Authorization code with PKCE - message sequence



Why PKCE works

```
verifier  = random(32+ bytes)
challenge = BASE64URL(SHA256(verifier))

/authorize?...&code_challenge=<challenge>&code_challenge_method=S256
/token     code=<code>&code_verifier=<verifier>
```

An intercepted authorization code is useless without the verifier retained by the initiating client.

What is bound to an authorization code

- Client identifier and exact registered redirect URI.
- Exact resource audience: the public API, not MCP or internal application routes.
- Granted scopes and workspace context.
- PKCE challenge, expiry, and single-use state.
- The user and consent decision.

Access token verification

1. Verify cryptographic integrity and accepted signing key.
2. Require issuer, exact audience, expiry, and not-before constraints.
3. Resolve the credential and principal as active, not revoked, and not expired.
4. Require the requested scope.
5. Reload current membership, role, team restriction, and resource state.

Delegated user versus OAuth application actor

Flow	Actor	Important rule
Authorization code	The consenting user, constrained by grant	Current user membership and role remain authoritative
Client credentials	The installed OAuth application principal	Never impersonate the installer or attribute writes to that human

Refresh-token rotation

```
token family: F

R1 --exchange--> R2 + A2     R1 becomes consumed
R2 --exchange--> R3 + A3     R2 becomes consumed
R1 --reused----> revoke F    possible theft or replay
```

Rotation narrows the value of a stolen old refresh token and makes reuse detectable.

> **OAuth is delegation, not permanent trust**
>
> A valid token proves an issued grant, but current account, workspace, installation, scope, and resource state can still revoke the operation. Token validity and business authorization are separate checks.

14. Webhook security and replay safety

A webhook is an HTTP request sent by another system. It is hostile input until authenticity, freshness, size, and identity are proven.

Why exact bytes matter

Signatures are computed over bytes, not a parsed JSON object. Parsing and re-encoding can change whitespace, key order, or number spelling and therefore change the signed message.

Why the durable inbox matters

- Providers retry deliveries when acknowledgements are slow or lost.
- Duplicate deliveries return success without duplicating the business effect.
- The worker can retry independently of the provider connection.
- The database records terminal, retryable, revoked, and malformed outcomes.

> **Queue payload rule**
> The queue carries only a stable provider key and inbox UUID. The worker loads the canonical encrypted receipt from PostgreSQL. Raw provider payloads and credentials do not travel through arbitrary queue messages.

14A. Outbound developer webhooks in detail

Outbound webhooks let an external application subscribe to FortyOne events. Delivery is treated as a durable state machine because endpoints fail, return ambiguous results, rotate secrets, and may receive duplicates.

From business event to HTTP delivery

Conceptual signature

```
timestamp = "1787942400"
delivery  = "evt_01..."
body      = exact JSON bytes

signed_payload = timestamp + "." + delivery + "." + body
signature      = HEX(HMAC_SHA256(endpoint_secret, signed_payload))

FortyOne-Timestamp: 1787942400
FortyOne-Delivery: evt_01...
FortyOne-Signature: v1=<signature>
```

The receiver reconstructs the same bytes, verifies HMAC in constant time, checks freshness, and deduplicates delivery ID.

Receiver verification pseudocode

```
if abs(now - timestamp) > allowedSkew { reject() }
expected := hmacSHA256(secret, timestamp + "." + delivery + "." + body)
if !hmac.Equal(expected, received) { reject() }
if store.Seen(delivery) { return success }
store.ProcessOnce(delivery, body)
```

Signature verification prevents tampering; delivery deduplication prevents duplicate effects.

Endpoint security and SSRF

A user-controlled webhook URL can become a server-side request forgery path. Endpoint registration and delivery therefore need scheme validation, DNS and address checks, redirect policy, connection timeouts, response limits, and revalidation when DNS may change.

Retry policy

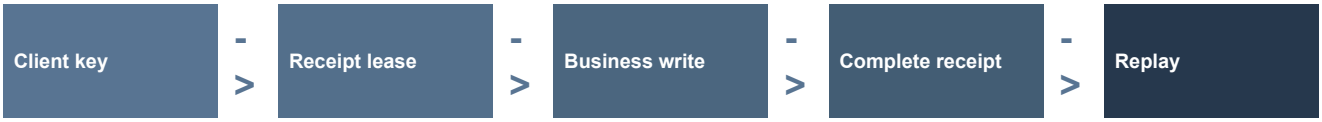
Outcome	Example	Action
Success	2xx	Complete; duplicate attempts must converge
Rate limited	429 + Retry-After	Respect bounded provider guidance
Transient	Timeout, selected 5xx	Exponential backoff with jitter and attempt ceiling
Client error	Most 4xx	Terminal unless explicitly classified otherwise
Disabled/revoked	Endpoint removed or secret invalidated	Stop delivery; require explicit operator action

Delivery guarantee

The practical contract is at-least-once delivery with stable identity, not magical exactly-once networking. Both sender and receiver use idempotency so duplicates are harmless.

15. Idempotency: safe retries

Networks fail ambiguously. A client may not know whether a request reached the server. Idempotency lets it retry one logical operation without creating duplicate business state.



Public story creation

1. The client generates one random Idempotency-Key per logical story creation.
2. The API binds the key to principal, credential identity, workspace, method, and operation.
3. It hashes the exact request bytes and attempts to acquire a receipt lease.
4. A completed identical request replays the stored safe response.
5. The same key with different bytes returns idempotency_key_reused.
6. A live concurrent request returns idempotency_in_progress with Retry-After.
7. A one-way external creation key plus database uniqueness closes the crash window after commit.

```
switch begin.State {
case Completed:  return replay(begin.Response)
case InProgress: return conflict("idempotency_in_progress")
case Conflict:   return conflict("idempotency_key_reused")
case New:        return createAndCompleteReceipt()
}
```

The receipt is a coordination state machine, not merely a cache.

The hard crash window

The story transaction can commit just before the receipt is marked complete. Database uniqueness on the derived external creation identity ensures recovery converges on the existing story instead of creating another one.

15A. Security feature inventory and threat map

Security is not one middleware. It is a collection of controls placed at the boundary where each threat can be stopped most reliably.

Threat	Primary controls	Failure prevented
Cross-workspace access	Typed actor, workspace policy, tenant SQL predicates, two-tenant tests	Reading or mutating another customer's data
Privilege escalation	Explicit principal kinds, current role checks, scope and team restrictions	A valid credential gaining unintended operations
Stolen database token	Show-once secret, HMAC digest storage, expiry and revocation	Recovering reusable API credentials from rows
Provider credential leak	Versioned envelope encryption, context binding, secret-safe logs	Using stored OAuth/provider tokens outside the vault
Webhook forgery	Bounded exact body, provider signature, timestamp and delivery identity	Unauthenticated external commands
Webhook replay	Durable inbox uniqueness, replay windows, generation fences	Repeating an already accepted effect
Duplicate mutation	Scoped idempotency receipts, request hash, database uniqueness	Creating the same logical resource twice
Session persistence after disable	Server-side session epoch and current account-state checks	Old cookies surviving revocation
OAuth code interception	Exact redirect URI, short expiry, single use, S256 PKCE	Exchanging a stolen authorization code
Refresh token replay	Rotation and family revocation	Long-lived reuse after token theft
SSRF through webhook URL	HTTPS policy, address validation, redirect and response limits	Reaching internal metadata or private services
Sensitive log leakage	Structured safe fields, bounded metadata, scanners and log tests	Secrets or customer payloads entering telemetry
Resource exhaustion	Body limits, rate limits, page caps, pool limits, bounded jobs	Memory, connection, or queue collapse
Supply-chain vulnerability	Pinned tooling, govulncheck, staticcheck, generated drift checks	Known reachable vulnerable code

Fail closed

When identity, role, key version, installation state, signature, or dependency coordination cannot be established safely, the operation is denied or made retryable. Unknown state is not permission. Production configuration also rejects public development secrets and insecure transport choices.

Security testing is mostly negative testing

- Same resource ID from another workspace.
- Valid token missing one scope or restricted to another team.
- Disabled principal, expired credential, revoked installation, or stale session epoch.
- Valid webhook body with wrong signature, old timestamp, duplicate identity, or oversized body.
- Same idempotency key with altered whitespace or payload.
- Secret values injected into errors or logs to confirm they never escape.

PART V

Integrations as a platform

Understand provider-neutral contracts, adapters, installation state, inbound and outbound delivery, and how to add a provider.

16. Control plane and provider adapters

An integration has shared lifecycle concerns and provider-specific behavior. The architecture keeps those separate.

Shared control plane	Provider adapter
Installation status and workspace binding	OAuth URLs and callback payloads
Encrypted credential lifecycle	Provider SDK calls
Identity linking and product authorization	Slack blocks, GitHub events, Figma payload shapes
Durable inbox/outbox and retries	Provider-specific rate limits and error classification
Audit and retention	Native messages, cards, forms, or threads

The adapter boundary

```
type MessagingProvider interface {
    VerifyInbound(ctx context.Context, request SignedRequest) (Delivery, error)
    Acknowledge(delivery Delivery) ProviderResponse
    Normalize(delivery Delivery) ([]Command, error)
    Deliver(ctx context.Context, installation Installation, message Message) (Receipt, error)
    Capabilities() Capabilities
}
```

Conceptual interface. Concrete provider SDK models remain inside adapters.

Capabilities instead of lowest-common-denominator design

Slack may support modal updates and ephemeral replies while another provider does not. The adapter declares capabilities such as threads, streaming, native forms, or ephemeral messages. Shared business logic does not pretend every provider is Slack.

> **The reuse boundary**
> GitHub and GitLab share code-host capabilities and story/workspace business rules. They do not share raw webhook types, OAuth endpoints, or API clients. Reuse the invariant, not accidental provider syntax.

17. An integration event lifecycle

The complete lifecycle is intentionally durable. External delivery is asynchronous, retryable, and linked to a stable product intent.

Installation and user identity are different

A Slack workspace installation identifies a provider account connected to a FortyOne workspace. A Slack user ID does not automatically become a FortyOne user. A verified identity link is required, and every product operation runs under that linked actor's current permissions.

Adding GitLab or another code host

1. Register a stable provider descriptor and supported capabilities.
2. Implement the code-host adapter interfaces.
3. Keep GitLab webhook and SDK types inside the GitLab adapter.
4. Reuse credential vault, installation, identity, inbox/outbox, and retry infrastructure.
5. Call existing stories and workspaces services rather than duplicating their rules.
6. Add contract tests plus approved live-provider tests.

18. Public developer platform

The browser-facing application API and the public developer API are different products. Only deliberately versioned routes under `/api/v1` are external contracts.

The contract includes more than happy paths

- Authentication schemes, scopes, and exact OAuth resource audience.
- Request body limits, content types, validation, and stable error codes.
- Opaque cursor pagination and bounded limits.
- Idempotency requirements and retry behavior.
- Rate-limit metadata and Retry-After.
- Webhook endpoint management and show-once signing secrets.

Generated clients

The OpenAPI source produces Go and TypeScript clients. Generation drift is checked, so server and SDK contracts cannot quietly diverge. Generated transport types remain at the boundary and are mapped to domain values.

```
client := sdk.NewClient(apiURL, sdk.WithBearerToken(token))

page, err := client.ListStories(ctx, workspaceID, sdk.ListOptions{Limit: 50})
for page.NextCursor != "" {
    page, err = client.ListStories(ctx, workspaceID,
        sdk.ListOptions{Limit: 50, Cursor: page.NextCursor})
}
```

Conceptual external-consumer flow: authenticate, read typed data, follow opaque cursors.

> **Compatibility rule**
> Clients must never decode cursor values or depend on database fields that are not in OpenAPI. The server may change cursor internals and persistence without breaking the public contract.

PART VI

Scalability, workers, and operations

Understand bounded work, leases, retries, observability, health, and why reliability is designed into persistence.

19. Bounded background work

A worker must assume there may be millions of rows, concurrent replicas, duplicate queue deliveries, crashes, and dependency failures. The safe pattern is bounded, resumable, and idempotent.

Keyset pagination

```
SELECT id, scheduled_at
FROM jobs
WHERE (scheduled_at, id) > (sqlc.arg(after_time), sqlc.arg(after_id))
ORDER BY scheduled_at, id
LIMIT sqlc.arg(batch_size);
```

Stable keyset order avoids deep OFFSET scans and supports resumable batches.

Why leases exist

A lease records temporary ownership. If a worker crashes, another worker can reclaim the item after expiry. Completion uses the claim token, so a stale worker cannot overwrite a newer worker's result.

State	Meaning	Recovery
Pending	Eligible but unclaimed	Any worker may claim
Leased	Owned until a deadline	Wait or reclaim after expiry
Completed	Terminal success	Duplicate delivery returns success
Retry	Transient failure	Backoff with bounded attempts
Terminal	Invalid, revoked, or exhausted	Operator or explicit reauthorization

> **Scaling principle**
> Do not make a job faster by loading everything into memory. Make it bounded, index-supported, restartable, and safe under concurrency first; then measure and optimize.

20. Pagination, rate limits, and backpressure

Scalability is often the art of bounding work. Requests, pages, bodies, retries, database pools, queue concurrency, and shutdown time all have explicit ceilings.

Opaque cursor contract

- A stable, unique order prevents duplicates or gaps while data changes.
- The cursor is signed, purpose-separated, and bound to filters, principal, workspace, and page size.
- The client returns it unchanged; it is not a public data structure.
- Limits are bounded even when the caller asks for more.

Rate limiting

Rate limits protect shared capacity and create predictable retry behavior. Rejected requests include authoritative `Retry-After`. Internal partition identities are never exposed in headers or logs.

Connection pools are concurrency limits

A pgx pool does not merely reuse connections. Its maximum size bounds concurrent database work from one process. Oversizing every replica can exhaust PostgreSQL; undersizing can create request queues. Production tuning requires observed latency, wait time, and database capacity.

> **Backpressure**
> When a downstream system is saturated, the correct response is usually to wait, reject with retry guidance, or queue bounded work. Spawning unlimited goroutines moves the failure into memory and connection exhaustion.

21. Observability and operational safety

Observability answers what the system is doing without exposing customer data or secrets. The API uses structured logs, request correlation, OpenTelemetry, health phases, and bounded-cardinality fields.

Signal	Answers	Safe example
Log	What happened in one operation?	operation, request ID, workspace ID when policy permits, error class
Trace	Where was time spent?	HTTP -> service -> repository -> PostgreSQL span chain
Metric	How often or how much?	latency histogram, queue age, retry count, pool wait
Readiness	Can this instance safely receive traffic?	phase plus bounded PostgreSQL and Redis checks

Never log

- Authorization headers, cookies, raw tokens, OAuth codes, or API keys.
- Credential vault plaintext or encryption keys.
- Raw webhook or customer payloads unless an explicitly secured retention design requires it.
- Exact idempotency keys or request bodies.

Deployment posture

FortyOne is currently a privately operated application. Unsupported public Compose, setup, licensing, and self-host support material was removed. Managed API and worker image/release paths remain. Self-hosting should return only as an explicitly supported product with documentation and operational capacity.

PART VII

Testing, changing, and learning

Turn the architecture into a practical workflow: tests by risk, a complete story trace, change steps, exercises, and references.

22. The testing strategy

Different tests answer different questions. A large end-to-end suite cannot replace fast policy tests, and mocks cannot prove PostgreSQL behavior.

Layer	Proves	Typical example
Domain	Pure rules and invariants	Allowed transition, normalization, finite values
Service	Use-case orchestration and policy	Authorization denied, transaction intent, error mapping
Repository	Real SQL and mapping	Tenant predicate, constraint, concurrency, null behavior
HTTP	Protocol contract	Body limit, unknown field, status, error shape, scope
Module slice	Layers cooperate	Create story through handler/service/repository
System	Runtime dependencies and journeys	API + PostgreSQL + Redis + worker recovery
Provider contract	Adapter semantics	Signature, normalization, retry classification
Race/fuzz/load	Concurrency, parser, and capacity risk	Token canonicalization, data race, p95 latency

Quality gates

```
make check-fast          # hermetic local gate
make sqlc-check          # generation drift and compile
make architecture-check  # dependency and direct-SQL rules
make staticcheck         # deeper static analysis
make security-check      # reachable vulnerabilities and gosec policy
make gitleaks-check      # secret scanning
make test-race           # concurrency safety
make test-fuzz           # governed parser/security fuzz targets
```

The recorded local snapshot passed these hermetic gates.

Evidence language

Say exactly what ran. A passing unit suite is not a passing migration test; a local startup is not a provider exercise; a generated query is not a proven production query plan.

23. Worked example: create a story through /api/v1

This trace ties the platform together. Read it once for the flow, then revisit the details after the earlier chapters.

Step 1 - transport and actor

```
actor, problem := actorFor(ctx, request.WorkspaceId, auth.ScopeStoriesWrite)
if problem == nil {
    problem = requireStoryWriter(actor)
}
if problem != nil {
    return createStoryFailure(ctx, problem, 0), nil
}
```

The public handler requires scope and an admitted story-writer principal.

Step 2 - exact bytes and idempotency

```
rawBody, ok := exactRequestBody(ctx)
key, err := idempotency.ParseKey(request.Params.IdempotencyKey)
scope, err := idempotency.NewScope(actor, idempotency.MethodPost, operation)
begin, err := receipts.Begin(ctx, scope, key, rawBody)
```

The exact body is contract-significant and never reconstructed for hashing.

Step 3 - map protocol to domain input

```
input := stories.CoreNewStory{
    Title: body.Title,
    Team: body.TeamId,
    Status: body.StatusId,
    Assignee: body.AssigneeId,
    LabelIDs: labelIDs,
    CreationKey: &derivedOneWayCreationKey,
}
```

OpenAPI types do not flow directly into the service or repository.

Step 4 - service policy

The service resolves current actor state, validates the team and related objective/key-result/status references, normalizes scheduling and estimate behavior, and constructs one typed mutation command.

Step 5 - transactional persistence

```
WithinTransaction(ctx, Serializable, func(tx pgx.Tx) error {
    queries := storysql.New(tx)
    authorizeCreate(queries, actor, workspace, team)
    sequence := queries.NextStorySequence(...)
    story := queries.CreateStoryMutation(...)
    queries.InsertAuthorizedStoryLabels(...)
    queries.UpsertStoryMutationActivity(...)
    queries.InsertStoryMutationEvent(...)
    return nil
})
```

The simplified transaction shows the invariant: approved state and its durable event commit together.

Step 6 - typed SQL

```
-- name: CreateStoryMutation :one
INSERT INTO public.stories (..., team_id, workspace_id, ...)
SELECT ..., sqlc.arg(team_id), sqlc.arg(workspace_id), ...
WHERE referenced_status_is_in_team
  AND referenced_assignee_is_allowed
  AND referenced_objective_is_in_workspace
RETURNING id, sequence_id, title, workspace_id, team_id, created_at;
```

Condensed from the real query. SQL protects reference scope as well as insert shape.

Step 7 - complete or recover

The handler stores the reviewed 201 response in the receipt. An identical retry replays it. If the process crashed after the story commit but before receipt completion, the derived external creation key and uniqueness rule lead the retry back to the same story.

> **What to notice**
>
> No single layer does everything. Safety comes from cooperating boundaries: transport limits, typed actors, service policy, repository transactions, SQL predicates, constraints, idempotency state, and durable events.

24. How to make a safe change

Example: add a filter to the story list

1. Locate the story route in the generated inventory and inspect its middleware.
2. Define the filter in transport-neutral story domain vocabulary.
3. Parse and bound the public query parameter in HTTP.
4. Pass the typed filter through the service port.
5. Add a named SQLC parameter and explicit SQL predicate.
6. Keep workspace, actor, team, deleted-state, and stable ordering predicates intact.
7. Regenerate SQLC and review all generated differences.
8. Test success, invalid input, cross-tenant denial, cursor binding, and repository behavior.
9. Update OpenAPI and regenerate SDKs if the route is public.
10. Run focused tests, then the required gates.

A complete-slice checklist

Concern	Question
Ownership	Which module owns this behavior and vocabulary?
Input	Is every body, string, list, page, and duration bounded?
Actor	Which principal kinds, scopes, workspace, team, and roles are allowed?
Data	Is the SQL named, typed, tenant-scoped, deterministic, and indexed?
Atomicity	Which state and event must commit together?
Retry	What happens after duplicate delivery or a crash?
Errors	Is the client contract stable and free of sensitive details?
Evidence	Which test layer proves each risk?

25. Hands-on learning exercises

These exercises are intentionally read-only until the last one. Complete them in order and explain each answer aloud.

Exercise 1 - trace a read

- Open the API inventory and choose GET story by ID.
- Find route, middleware, handler, service, repository, named SQL, and closest tests.
- Write down every place workspace or actor scope is enforced.

Exercise 2 - inspect generated SQLC

- Open a query under repository/queries and its generated method under repository/sqlc.
- Match every sqlc.arg to the generated parameter field.
- Find the handwritten row-to-domain mapping and explain why it is not generated.

Exercise 3 - simulate authorization

- Choose one human actor, one service account, and one OAuth application.
- For stories:write, evaluate principal kind, workspace, scope, team restriction, role, and resource.
- Change one input at a time and predict the denial code.

Exercise 4 - reason about a crash

- Assume story creation commits but the process stops before responding.
- Explain what the receipt lease, creation key, unique constraint, and replay response each do.

Exercise 5 - design a provider adapter

- Choose Microsoft Teams or another provider.
- List provider-specific ingress, identity, capabilities, rendering, and errors.
- List the shared installation, vault, inbox/outbox, authorization, and story behavior you would reuse.

Exercise 6 - make one small complete change

- Follow docs/onboarding/change-a-module.md.
- Keep the diff narrow, add risk-based tests, and run the required gates.

> **Mastery test**
> You understand the platform when you can predict where a change belongs, identify its trust boundaries, trace its data path, explain its failure recovery, and name the evidence needed before release.

26. Glossary

Term	Plain-language meaning
Actor	The authenticated principal plus credential restrictions used for authorization.
Adapter	Code that translates an external or infrastructure interface into an internal contract.
Capability	A focused behavior such as reading stories, delivering messages, or managing credentials.
Cursor	An opaque continuation token for a stable, bounded list.
Domain	Transport- and persistence-neutral business vocabulary and rules.
HMAC	A keyed one-way digest used to verify authenticity without storing a recoverable secret.
Idempotency	Executing one logical operation at most once despite retries.
Inbox	Durable record of an inbound external delivery and its processing state.
Invariant	A condition that must remain true across every valid state transition.
Lease	Temporary exclusive right to process work, with expiry for crash recovery.
Modular monolith	One deployable application with enforced internal capability boundaries.
Outbox	Durable record of an external side effect committed with business state.
pgx	The PostgreSQL driver, pool, and transaction runtime used by the Go application.
Principal	A user, service account, OAuth application, or other authenticated identity.
Repository	Persistence adapter that owns SQLC calls, transactions, mapping, and database error classification.
Scope	A credential-level capability restriction such as stories:read or stories:write.
SQLC	A generator that turns handwritten SQL into typed Go methods and structures.
Tenant	A workspace boundary whose data must not leak into another workspace.
Transaction	An all-or-nothing database unit that protects an invariant.
Webhook	An authenticated HTTP delivery from or to an external system.

27. Command reference

```
cd apps/server

make dev                # API with Air live reload
make worker             # worker process
make develop            # API without live reload

make sqlc-generate      # regenerate typed database code
make sqlc-check         # verify generated SQLC is current
make migration-check    # validate migration contracts
make architecture-check # enforce module and persistence boundaries
make check-fast         # main hermetic local gate

go test ./internal/modules/stories/...
go test -race ./internal/modules/stories/...
go test ./cmd/api ./internal/bootstrap/api
```

Run commands from apps/server unless the repository documentation says otherwise.

When live infrastructure is approved

```
make sqlc-vet           # requires SQLC_DATABASE_URL
make test-integration   # requires isolated TEST_DATABASE_URL and TEST_REDIS_URL
```

Environment-backed checks must fail clearly when dependencies are absent; they are never silently counted as passes.

> **Toolchain note**
>
> The module declares Go 1.25.0. Use the module-declared toolchain. The host's base 'go version' may be older while Go toolchain selection downloads or invokes the required version.

28. Source map for deeper study

These repository paths are the source references used to build this guide. Start with the first four, then follow the area you want to understand.

Topic	Repository path
Implementation status	docs/plans/api-modernization/00-implementation-status.md
First-hour onboarding	apps/server/docs/onboarding/first-hour.md
Engineering standards	apps/server/docs/architecture/standards.md
Generated inventory	apps/server/docs/inventory/api.md
API lifecycle	apps/server/docs/architecture/api-lifecycle.md
SQLC guide	apps/server/docs/database/sqlc.md
Story reads	apps/server/docs/database/stories-read.md
Story mutations	apps/server/docs/database/stories-mutations.md
Authorization	apps/server/docs/security/authorization.md
Credential vault	apps/server/docs/security/provider-credential-vault.md
Developer credentials	apps/server/docs/security/developer-credentials.md
Developer OAuth	apps/server/docs/security/developer-oauth.md
Idempotency	apps/server/docs/security/idempotency-receipts.md
Integration providers	apps/server/docs/integrations/providers.md
Webhook gateway	apps/server/docs/integrations/webhook-gateway.md
Public API v1	apps/server/docs/integrations/public-api-v1.md
OpenAPI source	apps/server/api/openapi/v1/openapi.yaml
External example	apps/server/examples/external-integration/README.md
Migration operations	apps/server/docs/database/migration-operations.md
Changing a module	apps/server/docs/onboarding/change-a-module.md

Key implementation examples

```
apps/server/internal/modules/apiv1/http/story_mutations.go
apps/server/internal/modules/stories/service/story_create.go
apps/server/internal/modules/stories/repository/mutations.go
apps/server/internal/modules/stories/repository/queries/mutations.sql
apps/server/internal/platform/authorization/policy.go
apps/server/internal/platform/idempotency/
apps/server/internal/platform/credentialvault/
apps/server/internal/platform/webhooks/
apps/server/internal/platform/integrations/
```

Trace these after completing the worked example.

THE PLATFORM IN ONE SENTENCE

A typed, secure, modular Go application where every request becomes an explicit actor decision, every database operation belongs to a capability, and every external side effect is durable and retry-safe.

Your next step

Trace one real GET request using the API inventory. Then trace the story-create example in this guide directly in the source. Understanding grows fastest when the diagram and the code are open side by side.

FortyOne API Platform Guide - implementation snapshot 28 August 2026